

Create and retrieve secrets from Azure Key Vault

In this exercise, you create an Azure Key Vault, store secrets using the Azure CLI, and build a .NET console application that can create and retrieve secrets from the key vault. You learn how to configure authentication, manage secrets programmatically, and clean up resources when finished.

Tasks performed in this exercise:

- Create Azure Key Vault resources
- Store a secret in a key vault using Azure CLI
- Create a .NET console app to create and retrieve secrets
- Clean up resources

Create Azure Key Vault resources and add a secret

In this section of the exercise you create the needed resources in Azure with the Azure CLI.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Use the [>_] button to the right of the search bar at the top of the page to create a new cloud shell in the Azure portal, selecting a **Bash** environment. The cloud shell provides a command line interface in a pane at the bottom of the Azure portal. If you are prompted to select a storage account to persist your files, select **No storage account required**, your subscription, and then select **Apply**.

Note: If you have previously created a cloud shell that uses a *PowerShell* environment, switch it to **Bash**.

3. In the cloud shell toolbar, in the **Settings** menu, select **Go to Classic version** (this is required to use the code editor).
4. Create a resource group for the resources needed for this exercise. If you already have a resource group you want to use, proceed to the next step. Replace **myResourceGroup** with a name you want to use for the resource group. You can replace **eastus** with a region near you if needed.

In this **Cloudslice** lab, your **Resource Group** has already been created (**myResourceGrouplod58094841**) and you may safely skip this step.

TypeCopy

```
az group create --name myResourceGroup --location eastus
```

5. Many of the commands require unique names and use the same parameters. Creating some variables will reduce the changes needed to the commands that create resources. Run the following commands to create the needed variables. Replace **myResourceGroup** with the name you're using for this exercise. If you changed the location in the previous step, make the same change in the **location** variable.

TypeCopy

```
resourceGroup=myResourceGrouplod58094841
```

```
location=eastus
```

```
keyVaultName=mykeyvaultname58094841
```

6. Run the following command to get the name of the key vault and record the name. You need it later in the exercise.

TypeCopy

```
echo $keyVaultName
```

7. Run the following command to create an Azure Key Vault resource. This can take a few minutes to run.

TypeCopy

```
az keyvault create --name $keyVaultName \  
--resource-group $resourceGroup --location $location
```

Assign a role to your Microsoft Entra user name

To create and retrieve a secret, assign your Microsoft Entra user to the **Key Vault Secrets Officer** role. This gives your user account permission to set, delete, and list secrets. In a typical scenario you may want to separate the create/read actions by assigning the **Key Vault Secrets Officer** to one group, and **Key Vault Secrets User** (can get and list secrets) to another.

1. Run the following command to retrieve the **userPrincipalName** from your account. This represents who the role will be assigned to.

TypeCopy

```
userPrincipal=$(az rest --method GET --url https://graph.microsoft.com/v1.0/me \  
--headers 'Content-Type=application/json' \  
--query userPrincipalName --output tsv)
```

2. Run the following command to retrieve the resource ID of the key vault. The resource ID sets the scope for the role assignment to a specific key vault.

TypeCopy

```
resourceID=$(az keyvault show --resource-group $resourceGroup \
--name $keyVaultName --query id --output tsv)
```

3. Run the following command to create and assign the **Key Vault Secrets Officer** role.

TypeCopy

```
az role assignment create --assignee $userPrincipal \
--role "Key Vault Secrets Officer" \
--scope $resourceID
```

Next, add a secret to the key vault you created.

Add and retrieve a secret with Azure CLI

1. Run the following command to create a secret.

TypeCopy

```
az keyvault secret set --vault-name $keyVaultName \
--name "MySecret" --value "My secret value"
```

2. Run the following command to retrieve the secret to verify it was set.

TypeCopy

```
az keyvault secret show --name "MySecret" --vault-name $keyVaultName
```

This command returns some JSON. The last line contains the password in plain text.

jsonTypeCopy

```
"value": "My secret value"
```

Create a .NET console app to store and retrieve secrets

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in the cloud shell.

Tip: Resize the cloud shell to display more information, and code, by dragging the top border. You can also use the minimize and maximize buttons to switch between the cloud shell and the main portal interface.

1. Run the following commands to create a directory to contain the project and change into the project directory.

TypeCopy

```
mkdir keyvault
```

```
cd keyvault
```

2. Create the .NET console application.

TypeCopy

```
dotnet new console
```

3. Run the following commands to add the **Azure.Identity** and **Azure.Security.KeyVault.Secrets** packages to the project.

TypeCopy

```
dotnet add package Azure.Identity
```

```
dotnet add package Azure.Security.KeyVault.Secrets
```

Add the starter code for the project

1. Run the following command in the cloud shell to begin editing the application.

TypeCopy

```
code Program.cs
```

2. Replace any existing contents with the following code. Be sure to replace **YOUR-KEYVAULT-NAME** with your actual key vault name.

csharpTypeCopy

```
using Azure.Identity;
```

```
using Azure.Security.KeyVault.Secrets;
```

```
// Replace YOUR-KEYVAULT-NAME with your actual Key Vault name
```

```
string KeyVaultUrl = "https://YOUR-KEYVAULT-NAME.vault.azure.net/";
```

```
// ADD CODE TO CREATE A CLIENT
```

// ADD CODE TO CREATE A MENU SYSTEM

// ADD CODE TO CREATE A SECRET

// ADD CODE TO LIST SECRETS

3. Press **ctrl+s** to save your changes.

Add code to complete the application

Now it's time to add code to complete the application.

1. Locate the **// ADD CODE TO CREATE A CLIENT** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

// Configure authentication options for connecting to Azure Key Vault

```
DefaultAzureCredentialOptions options = new()
```

```
{
```

```
    ExcludeEnvironmentCredential = true,
```

```
    ExcludeManagedIdentityCredential = true
```

```
};
```

// Create the Key Vault client using the URL and authentication credentials

```
var client = new SecretClient(new Uri(KeyVaultUrl), new  
DefaultAzureCredential(options));
```

2. Locate the **// ADD CODE TO CREATE A MENU SYSTEM** comment and add the following code directly after the comment. Be sure to review the code and comments.

```
csharpTypeCopy
```

```
// Main application loop - continues until user types 'quit'
```

```
while (true)
```

```
{
```

```
// Display menu options to the user
```

```
Console.Clear();
```

```
Console.WriteLine("\nPlease select an option:");
```

```
Console.WriteLine("1. Create a new secret");
```

```
Console.WriteLine("2. List all secrets");
```

```
Console.WriteLine("Type 'quit' to exit");
```

```
Console.Write("Enter your choice: ");
```

```
// Read user input and convert to lowercase for easier comparison
```

```
string? input = Console.ReadLine()?.Trim().ToLower();
```

```
// Check if user wants to exit the application
```

```
if (input == "quit")
```

```
{
```

```
    Console.WriteLine("Goodbye!");
```

```
    break;
```

```
}
```

```
// Process the user's menu selection
```

```
switch (input)
```

```
{
```

```
    case "1":
```

```

        // Call the method to create a new secret
        await CreateSecretAsync(client);

        break;
    case "2":
        // Call the method to list all existing secrets
        await ListSecretsAsync(client);

        break;
    default:
        // Handle invalid input
        Console.WriteLine("Invalid option. Please enter 1, 2, or 'quit.'");

        break;
    }
}

```

3. Locate the **// ADD CODE TO CREATE A SECRET** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

```

async Task CreateSecretAsync(SecretClient client)
{
    try
    {
        Console.Clear();

        Console.WriteLine("\nCreating a new secret...");

        // Get the secret name from user input
        Console.Write("Enter secret name: ");

        string? secretName = Console.ReadLine()?.Trim();

        // Validate that the secret name is not empty
    }
}

```

```
if (string.IsNullOrEmpty(secretName))
{
    Console.WriteLine("Secret name cannot be empty.");
    return;
}

// Get the secret value from user input

Console.Write("Enter secret value: ");
string? secretValue = Console.ReadLine()?.Trim();

// Validate that the secret value is not empty
if (string.IsNullOrEmpty(secretValue))
{
    Console.WriteLine("Secret value cannot be empty.");
    return;
}

// Create a new KeyVaultSecret object with the provided name and value
var secret = new KeyVaultSecret(secretName, secretValue);

// Store the secret in Azure Key Vault
await client.SetSecretAsync(secret);

Console.WriteLine($"Secret '{secretName}' created successfully!");
Console.WriteLine("Press Enter to continue...");
Console.ReadLine();
}

catch (Exception ex)
```

```

{
    // Handle any errors that occur during secret creation

    Console.WriteLine($"Error creating secret: {ex.Message}");
}
}

```

4. Locate the **// ADD CODE TO LIST SECRETS** comment and add the following code directly after the comment. Be sure to review the code and comments.

csharpTypeCopy

async Task ListSecretsAsync(SecretClient client)

```

{
    try
    {
        Console.Clear();

        Console.WriteLine("Listing all secrets in the Key Vault...");
        Console.WriteLine("-----");

        // Get an async enumerable of all secret properties in the Key Vault
        var secretProperties = client.GetPropertiesOfSecretsAsync();
        bool hasSecrets = false;

        // Iterate through each secret property to retrieve full secret details
        await foreach (var secretProperty in secretProperties)
        {
            hasSecrets = true;

            try
            {
                // Retrieve the actual secret value and metadata using the secret name
                var secret = await client.GetSecretAsync(secretProperty.Name);
            }
            catch { }
        }
    }
}

```

```

        // Display the secret information to the console

        Console.WriteLine($"Name: {secret.Value.Name}");

        Console.WriteLine($"Value: {secret.Value.Value}");

        Console.WriteLine($"Created: {secret.Value.Properties.CreatedOn}");

        Console.WriteLine("-----");
    }

    catch (Exception ex)
    {
        // Handle errors for individual secrets (e.g., access denied, secret not found)

        Console.WriteLine($"Error    retrieving    secret    '{secretProperty.Name}':
{ex.Message}");

        Console.WriteLine("-----");
    }
}

// Inform user if no secrets were found in the Key Vault

if (!hasSecrets)
{
    Console.WriteLine("No secrets found in the Key Vault.");
}

}

catch (Exception ex)
{
    // Handle general errors that occur during the listing operation

    Console.WriteLine($"Error listing secrets: {ex.Message}");

}

```

```
Console.WriteLine("Press Enter to continue...");  
  
Console.ReadLine();  
  
}
```

5. Press **ctrl+s** to save the file, then **ctrl+q** to exit the editor.

Sign into Azure and run the app

1. In the cloud shell, enter the following command to sign into Azure.

TypeCopy

```
az login
```

You must sign into Azure - even though the cloud shell session is already authenticated.

Note: In most scenarios, just using *az login* will be sufficient. However, if you have subscriptions in multiple tenants, you may need to specify the tenant by using the *--tenant* parameter. See [Sign into Azure interactively using Azure CLI](#) for details.

2. Run the following command to start the console app. The app will display the menu system for the application.

TypeCopy

```
dotnet run
```

3. You created a secret at the beginning of this exercise, enter **2** to retrieve and display it.
4. Enter **1** and enter a secret name and value to create a new secret.
5. List the secrets again to view your new addition.

Enter **quit** when you are finished with the application.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.
2. Navigate to the resource group you created and view the contents of the resources used in this exercise.
3. On the toolbar, select **Delete resource group**.

4. Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.